



SWBWA: A Highly Efficient NGS Aligner on the New Sunway Architecture

Lifeng Yan¹, Zekun Yin^{1(✉)}, Qixin Chang¹, Tong Zhang¹, Zhisong Wang¹, Xiaohui Duan¹, Bertil Schmidt², and Weiguo Liu^{1(✉)}

¹ School of Software, Shandong University, Jinan, China
{lifeng.yan,cqx,tong.z,wzssdu}@mail.sdu.edu.cn,
{zekun.yin,sunrise.duan,weiguo.liu}@sdu.edu.cn

² Institute for Computer Science, Johannes Gutenberg University, Mainz, Germany
bertil.schmidt@uni-mainz.de

Abstract. Sequence alignment is a crucial step in next-generation sequencing data analysis. However, most sequence aligners face performance challenges due to high computational complexity and extensive random memory access patterns, making them a significant bottleneck in the overall analysis pipeline, such as the industry gold standard BWA-MEM. The next-generation Sunway platform, with its high computational power and unique heterogeneous architecture, presents new opportunities for enhancing the efficiency of sequence alignment. In this work, we introduce SWBWA, a high-accuracy and high-performance sequence aligner designed for the new Sunway architecture. By redesigning the parallel framework tailored for Sunway, performing software prefetching optimization, vectorizing the striped Smith-Waterman algorithm, and addressing memory access bottlenecks in bigshare mode, SWBWA achieves a 330× speedup over the single-threaded unoptimized version. Additionally, SWBWA running on a Sunway workstation can achieve 1.2–1.4× speedups compared to BWA-MEM running on a dual-socket 48-core x86 server, while ensuring nearly identical output. The source code is publicly available at <https://github.com/RabbitBio/SWBWA>.

Keywords: Sequencing read alignment · Bioinformatics · HPC · Sunway architecture

1 Introduction

1.1 Architecture of the New Sunway Platform

The SW26010-Pro, which is the processor of the new Sunway platform, features six Core Groups (CGs), each with a Management Processing Element (MPE) and 64 Compute Processing Elements (CPEs), providing strong support for complex parallel workloads [20]. The Sunway memory hierarchy consists of three segments: *shared*, *private*, and *cross*. The shared segment provides a common memory space within a CG, accessible by both the MPE and CPEs. The private segment is exclusive to each CPE, ensuring localized memory access. The cross

segment enables interleaved addressing across all six CGs, allowing synchronized memory access among them. Additionally, each CPE is equipped with 256KB of high-speed Local Data Memory (LDM), which can be manually managed by the user or partially configured as hardware-controlled cache to optimize computational efficiency. Typically, each CG runs a separate process bound to its MPE, with inter-process communication handled via the Message Passing Interface (MPI). Alternatively, we can also employ the bigshare mode, launching a single process only on the MPE of CG0 to manage all 384 CPEs on the chip, eliminating the need for MPI within a single node or workstation.

The SW26010-Pro processor is initially designed for the Sunway supercomputing platform. In recent years, small workstations equipped with a single SW26010-Pro have emerged, facilitating the use of the Sunway architecture beyond large-scale supercomputers. By combining Sunway workstations with bigshare mode, multi-process configurations are no longer needed, simplifying the adaptation and deployment of software originally developed for single-node x86 platforms. This approach promotes the wider adoption of the Sunway platform and contributes to the development of its software ecosystem, enabling more accessible and versatile applications beyond supercomputing environments.

1.2 Sequencing Read Alignment

Recent years have seen a tremendous increase in the volume of data generated in the life sciences, especially propelled by the rapid progress of next-generation sequencing (NGS) technologies with corresponding high-throughput technologies producing hundreds of millions of short DNA fragments (called reads) in a single run. Alignment of sequencing reads to a given reference genome (also called read mapping) is a computationally intensive step within common NGS data analysis pipelines, posing significant challenges in terms of memory consumption and computational burden when processing large-scale datasets [14]. It plays a crucial role in identifying genomic variations, reconstructing genomes, analyzing evolutionary relationships, personalized medicine, and metagenomics [6]. Currently, most alignment tools employ a seed-and-extend strategy to balance efficiency and accuracy. In this approach, the seeding phase rapidly identifies candidate alignment locations by matching short sequence fragments (seeds) typically consisting of k -mers (short DNA substrings of length k), while the extension phase performs a more precise alignment to validate these candidates and compute full alignment scores.

Minimizers [10], as employed in Minimap2 [8], select the smallest hash value within a sliding window to identify seeds in order to reduce storage and computational overhead, thereby improving alignment efficiency. Mashmap [5] combines Minimizers and MinHash to propose a new method for computing Jaccard similarity, further reducing the number of k -mers to enhance efficiency. Strobealign [13] incorporates strobemers [12] and syncmers [3], which construct seeds from multiple spaced k -mers, enhancing robustness in sequence matching. Furthermore, to optimize the extension phase, particularly the computationally expensive Smith-Waterman algorithm [16], various acceleration techniques have been

developed, including SIMD vectorization [1] and Hardware-based solutions such as FPGA [11] and GPU [9, 15] accelerators, significantly improving computational efficiency.

The powerful computational capabilities of the Sunway platform provide a strong foundation for addressing alignment performance bottlenecks. On Sunway TaihuLight [4], which is based on the previous-generation SW26010 processor, several alignment tools such as S-Aligner [2] and FMapper [18] have been successfully deployed. These tools adopt an “all-mode” alignment strategy, which reports all possible mapping locations that satisfy a given constraint, such as an edit distance threshold. While computationally intensive, this approach aligns well with the architecture of the SW26010 processor. However, due to limitations in main memory, LDM, and cache size, the SW26010 processor struggled to efficiently deploy software based on any-best mode algorithms, such as the industry-standard BWA-MEM [7], which selects only the best or a few top-scoring alignments for each sequencing read to reduce computational overhead. With the architectural advancements of the SW26010-Pro processor, including automatic data caching and increased main memory capacity, the efficient deployment of BWA-MEM on the new Sunway platform has now become feasible.

1.3 Contributions

In this work, we port and optimize BWA-MEM on the new Sunway platform, leveraging several optimization strategies to fully exploit the heterogeneous many-core architecture. Our key contributions include:

- Design of a parallel framework and memory management strategy to maximize the utilization of all CPEs and LDM, achieving a $17\times$ speedup compared to the single-threaded MPE version.
- Optimization of FM-index accesses through software prefetching and redundant computation reduction, combined with the vectorization of the striped Smith-Waterman (SSW) algorithm using Sunway’s SIMD instructions, significantly accelerating both the seeding and extension phases.
- Development of an all-CPE-based dynamic task partitioning framework for the bigshare mode to achieve better offloading and enabling overlapping of compute- and memory-intensive tasks, achieving an additional 30% performance improvement.
- Reducing the iCache and CPE private memory access problems in bigshare mode, leading to a $2\times$ performance boost.

Overall, we propose SWBWA, an efficient NGS read alignment tool optimized for the Sunway workstation platform with a single SW26010-Pro processor, fully utilizing all CGs without requiring MPI. Compared to BWA-MEM running on a dual-socket 48-core x86 server, SWBWA achieves an average $1.3\times$ speedup while delivering nearly identical results.

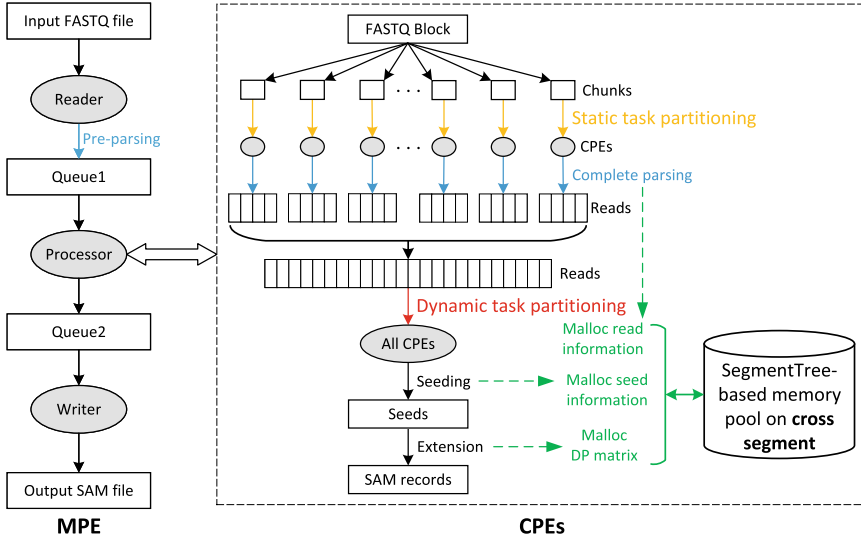


Fig. 1. Workflow of SWBWA. Blue indicates the parsing strategy, yellow and red represent task partitioning strategies, and green shows CPE memory management. (Color figure online)

2 Design of SWBWA

2.1 Parallelization Strategies for the Sunway Architecture

BWA-MEM employs a single-producer, multiple-consumer parallel strategy, where a producer thread parses the raw input file to generate FASTQ records, multiple consumer threads dynamically fetch and process data using an atomic index, and a writer thread outputs data in SAM format. As the number of threads increases, the performance of this strategy is progressively limited by the producer, particularly on the Sunway platform, where the single-core frequency is relatively low. To address this issue, SWBWA adopts the efficient FASTQ parsing method proposed in SWQC [19] for the Sunway platform. The MPE is only responsible for file I/O and simple block partitioning, while the entire parsing task is offloaded to multiple CPEs for parallel execution. We employ a hybrid static-dynamic task allocation strategy to ensure data order while maximizing multi-CPE parallelism. Figure 1 illustrates the workflow and parallelization strategy of SWBWA. The following subsections provides further details including asynchronous I/O and efficient task partitioning.

SWBWA initializes three threads—Reader, Processor, and Writer—on the MPE to handle file reading, CPE launch, and file writing, respectively. These threads exchange data over two shared queues, *Queue1* and *Queue2*. Additionally, the entire workflow follows a FASTQ block-based streaming approach, allowing memory usage to be controlled by adjusting the block size and queue capacity. When the program starts, the Reader thread continuously reads input

data in blocks (default size: 32 MB), performing pre-parsing while ensuring that no FASTQ record spans across two blocks by adjusting the tail pointer (illustrated in blue in Fig. 1). The Processor thread retrieves FASTQ blocks from *Queue1* and launches multiple CPEs for parallel parsing. In this step, SWBWA applies static task partitioning based on block size, where each CPE handles a designated chunk (default size: $32\text{ MB}/386 \approx 85\text{ KB}$) according to its index (illustrated in orange in Fig. 1). Each CPE then fully parses its assigned chunk, formats it into FASTQ records, and merges them into a large array. After formatting, all CPEs utilize a global atomic variable to dynamically fetch FASTQ records from this array, enabling dynamic task allocation (illustrated in red in Fig. 1). Then, each FASTQ record undergoes the seeding and extension phases to generate SAM information. To ensure high accuracy, we directly adopt the corresponding algorithms from BWA-MEM, using the FM-index for seed searching and the SSW algorithm for precise alignment, as detailed in the BWA-MEM paper [7]. Finally, the CPEs organize the SAM records into an ordered data block based on their index and pass it to the Writer thread for file output.

For task partitioning, SWBWA employs a hybrid static-dynamic strategy to ensure both efficiency and sequence order. First, SWBWA applies a static task partitioning strategy based on chunk size and CPE ID during data parsing. This approach is adopted because the complexity of FASTQ parsing is directly proportional to the chunk size, ensuring load balance while avoiding the overhead of maintaining atomic variables. In contrast, the computational complexity of the seeding and extension phases varies significantly across different FASTQ records. For instance, some low-abundance reads may yield zero seeds, while others may generate multiple matching locations (such as those stemming from repetitive genomic regions). Consequently, applying static partitioning would result in severe load imbalance, making dynamic task allocation essential.

Additionally, we enable the bigshare mode on the Sunway workstation, allowing a single process to manage all 384 CPEs. This approach eliminates the need for MPI and avoids synchronization and communication challenges when multiple processes write a single ordered file. However, the native bigshare mode on Sunway encounters certain performance issues, which are particularly pronounced in SWBWA. We provide our optimization details of the bigshare mode in Sect. 2.3.

2.2 Software Prefetching and SIMD Acceleration

Similar to BWA-MEM, in the seeding phase, SWBWA utilizes the FM-index data structure for efficient string matching and subsequent extension, identifying super maximal exact matches (SMEMs) for each read. Identified SMEMs are then processed using a B-tree-based chaining method, generating the final seeds. In this phase, a key performance bottleneck is random accesses to the FM-index. To address this issue, we employ software prefetching strategies. Specifically, during forward extension, match positions computed in one iteration are accessed in the next, enabling effective prefetching. Likewise, it can also be applied to backward extensions by leveraging data dependencies. Additionally, BWA-MEM

computes all four possible bases (ACGT) during string extension, but only one is used in the final result, leading to computational redundancy. We optimize the extension process without modifying the index structure, particularly in forward extension, where the computation is reduced to one-fourth of the original amount.

In the extension phase, SWBWA employs the SSW algorithm to accurately extend the seeds, generate CIGAR information, and then produce output in SAM format. Vectorized implementations of the SSW algorithm are a common and effective method for accelerating pairwise alignment computations based on dynamic programming [1]. During the implementation of a SIMD-optimized algorithm on Sunway CPEs, we encounter several challenges. The first challenge is the absence of a critical “compare-select” instruction in the CPE’s fixed-point instruction set. To overcome this, we emulate this operation using a combination of comparison, logical operations, and addition instructions. The second challenge is that CPEs only support int32-granularity instructions, lacking native support for int16 and int8 operations. To address this, we convert int8/int16 data into fp16 for computation, doubling the utilization of vector registers. We manually implement non-overflowing addition and subtraction within the 0–255/65535 range to compensate for the lack of native int8/int16 support. Additionally, we store both the dynamic programming matrix and the scoring matrix of the SSW process in the CPE’s high-speed LDM, significantly reducing memory access times.

With these optimizations, we mitigate random memory access delays in the seeding phase and accelerate SSW computation for the extension phase. However, fundamental bottlenecks remain unchanged: seeding is still memory-bound, while extension remains compute-bound. In BWA-MEM’s block-based processing, all threads first complete seeding, synchronize with a barrier, and then proceed with extensions. Evidently, this leads to underutilized compute resources in the first phase and inefficient memory bandwidth usage in the second. SWBWA eliminates this barrier, allowing both phases to overlap, maximizing hardware resource utilization.

2.3 Optimizing Sunway Bigshare Mode

We avoid using MPI in single-node environments by adopting the bigshare mode. In this mode, the process is bound to CG0, with all code segments and program memory allocated in the main memory of CG0 (Fig. 2). However, this setup introduces performance issues related to iCache misses and CPE private space accesses. The following subsections discuss these challenges and present corresponding solutions.

Mitigating iCache Contention. The SW26010-Pro processor lacks an L2 iCache and is equipped only with a 32KB L1 iCache; thus, once an iCache miss occurs, instructions can only be fetched from main memory. In bigshare mode, processes are bound to CG0, and all CPEs across the six CGs share a single code

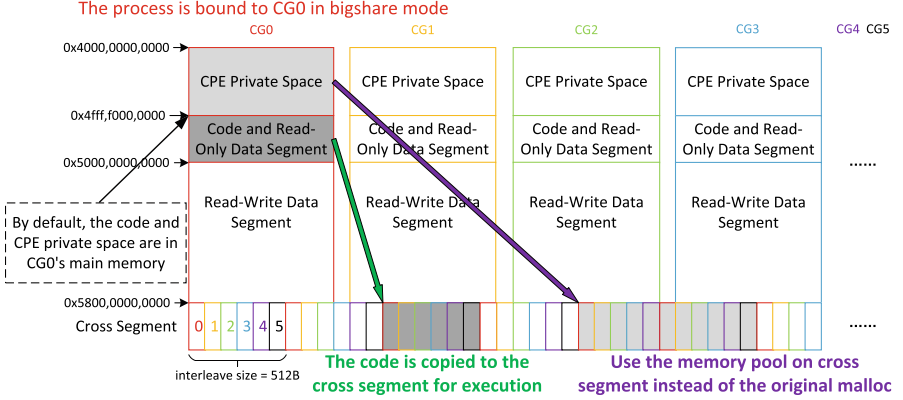


Fig. 2. Addressing performance bottlenecks in bigshare mode.

segment stored in CG0's main memory. This arrangement means that whenever an iCache miss occurs, all CPEs must compete for CG0's memory bandwidth, while the memory resources of the other five CGs remain underutilized, drastically reducing instruction fetch efficiency. To verify this, we have conducted a performance evaluation of iCache behavior using a straightforwardly parallelized version of SWBWA (see Table 1). The results show that when running on a single CG, due to extensive macro expansion and redundant instruction duplication, iCache miss overhead accounts for 16% of the total execution time. In bigshare mode, this issue is further exacerbated, with overall iCache miss overhead increasing to 31%, and some functions such as `bwa_gen_cigar2` reaching as high as 85%. As a result, the bigshare mode version (6 CGs) achieves only a $2\times$ speedup compared to the single CG version.

Figure 2 illustrates the memory space organization of the Sunway platform. The address range from 0x4000,0000,0000 to 0x5800,0000,0000 corresponds to the private space within each CG, which is not shared across CGs. This region includes the CPE private space, code segment, and data segment of each CG. Beyond 0x5800,0000,0000 is the cross segment, which employs an interleaved addressing scheme (interleave size is 512B) across six CGs and is shared among CGs. More formally, the range [0B, 512B) in the cross segment is physically mapped to CG0, [512B, 1024B) to CG1, [1024B, 1536B) to CG2, and so on, as illustrated in Fig. 2. Therefore, utilizing the cross segment can effectively alleviate memory access imbalance among CGs.

To address the iCache performance issues under the bigshare mode, SWBWA proposes a cross segment instruction loading strategy: copying the code segment from the main memory of CG0 to the cross segment for execution, as illustrated in green in Fig. 2. Specifically, at the beginning of the program, SWBWA first retrieves the starting address of the code segment (typically beginning with 0x4FFF, denoted as `oldTextSegAddr`) along with its size. It then allocates an equivalent memory space (beginning with 0x5800, denoted as `newTextSegAddr`)

Table 1. iCache performance evaluation.

Function	Single CG			Bigshare Mode (6 CGs)		
	Total ^a	iCache ^b	Ratio(%)	Total	iCache	Ratio(%)
run_alignment	936.9G	150.0G	16.01	492.6G	152.6G	30.98
seeding	689.1G	118.1G	17.14	391.8G	94.4G	24.09
collect_intv	284.5G	17.8G	6.26	159.0G	47.8G	30.06
btree_chain	188.8G	16.3G	8.63	116.4G	14.2G	12.20
mem_chain2aln	144.9G	59.4G	40.99	63.4G	11.8G	18.61
mem_sort_dedup	12.4G	4.9G	39.52	4.1G	2.3G	56.10
extension	247.7G	31.3G	12.64	129.3G	73.7G	57.00
mem_matesw2	196.8G	7.5G	3.81	71.1G	11.6G	16.32
mem_sort	34.1G	2.7G	7.92	15.5G	2.3G	14.84
bwa_gen_cigar2	15.4G	3.7G	24.03	12.9G	11.0G	85.27

^a Total execution cycles counted using `penv_slave0_cycle_count`

^b iCache miss penalty cycles counted using `penv_slave2_l1ic_misstime_count`

in the cross segment and manually assigns executable permissions. Subsequently, we copy the code to the cross segment and start the CPE by redirecting its program counter (PC) to the new CPE function address. Since this strategy changes the storage location of the code and the CPE startup mechanism, the following issues need to be addressed:

1. Some native CPE APIs (such as spawning and joining operations) may become invalid. To address this issue, we reimplement the CPE startup and shutdown functions using `__uncached` atomic variables, ensuring that the MPE and CPE can perform task scheduling and synchronization properly.
2. Although the CPE code is copied to the cross segment for execution, the function jump addresses in the original global offset table (GOT) still point to the old segment, causing function call failures due to excessive jump offsets. To resolve this, we first extract the instructions requiring relocation and the corresponding GOT items (denoted as `badGotItems`) from the relocation section (`.rela.text1`). We then copy a new GOT to the cross segment and update `badGotItems` by applying an offset (`segOffset = newTextSegAddr - oldTextSegAddr`) to redirect them to the new code segment. Additionally, the global pointer (GP) must also be adjusted by `segOffset` to ensure the correct resolution of all global variables and function jump addresses.
3. On the Sunway platform, to enhance performance and avoid thread synchronization overhead, the heap used by the CPE `malloc` operation and the I/O buffer for `fprintf` are typically private and stored in the CPE's thread local storage (TLS). However, instructions and function jumps involving TLS are generally addressed through control and status register (CSR) rather than GP, making the above strategy ineffective. To address this, we copy the relevant CPE private spaces to the cross segment and update the respective CSR

to point to the new space. Additionally, we update the corresponding code sections using relocation information from the `.rela.slave_tls` section.

In summary, by adopting the cross segment instruction loading strategy, SWBWA effectively mitigates iCache contention and reduces the performance loss caused by iCache misses under the bigshare mode. It is important to note that this strategy does not reduce the number of CPE iCache misses but instead balances instruction access requests across all CGs, thereby reducing the cost of each iCache miss and improving performance.

Efficient CPE Memory Access. Similar to the code segment, in bigshare mode, all program memory is allocated in the main memory of CG0, including the reference genome (allocated via the MPE `malloc` interface) and the intermediate results of the CPEs (allocated via the CPE `malloc` interface). The random access to the reference genome occurs during the seeding phase and represents the largest performance bottleneck in the entire program. Additionally, SWBWA involves numerous memory allocation requests by CPEs (as shown in green in Fig. 1), most of which are of unknown size and cannot be preallocated by the MPE in advance. The operations on these CPE-allocated memory span the entire alignment process and significantly impact CPE performance. As a result, we face a challenge similar to the previous iCache issue, with memory accesses concentrated in CG0, wasting bandwidth in other CGs. Therefore, redirecting CPE malloc-allocated addresses from private space to the cross segment is essential. To verify this, we compare the read/write performance on these two types of spaces (as shown in Fig. 3). The results indicate that when randomly reading/writing a memory space exceeding 128KB (the data cache size), the cross segment achieves around a $1.6\times$ speedup compared to the CPE private space. When the accessed memory is within the cache size, the cross segment is still faster.

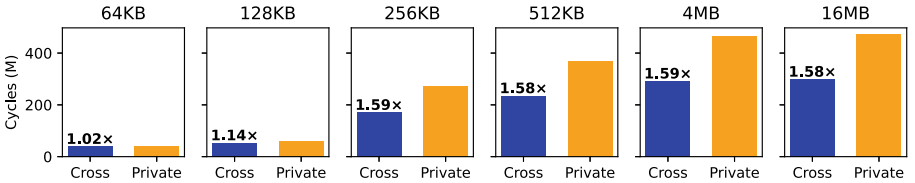


Fig. 3. Performance comparison for accessing different memory types in bigshare mode.

The Sunway platform provides the `_sw_xmalloc` interface for MPE, enabling direct allocation in the cross segment. By simply passing the `-xmalloc` parameter to the loader, all MPE `malloc` calls are automatically replaced with `_sw_xmalloc` in bigshare mode. Therefore, efficient access to the reference genome is not a major challenge. However, the CPE lacks APIs for allocating cross segment

space directly, requiring manual memory management to handle operations in the cross segment. SWBWA implements a memory pool using the cross segment, redirecting CPE malloc-allocated space to the cross segment (as shown in the purple in Fig. 2). To efficiently manage the memory pool while minimizing source code modifications, we redesign the `malloc`, `realloc`, and `free` functions using a non-recursive segment tree. Initially, the MPE preallocates a large block from cross segment (with an average of 8MB per CPE) as the memory pool, which consists of elements with only power-of-two sizes. Accordingly, when a CPE calls `malloc`, the requested size is rounded up to the nearest power-of-two before accessing the memory pool. Then, SWBWA efficiently locates the first available space in the segment tree corresponding to the adjusted size, returns the address, and updates the tree. Similarly, during `free`, it identifies the corresponding segment tree based on the address and updates it accordingly. In this way, the CPE private space, originally allocated by `malloc` and bound to CG0, is redirected to the cross segment, significantly improving memory access efficiency under bigshare mode.

3 Performance Evaluation

We first analyze the performance of different versions of SWBWA, demonstrating the improvements achieved by our proposed optimization techniques. We then compare the performance of SWBWA on the Sunway workstation with the latest version of BWA-MEM (v0.7.18) on a 48-core x86 server using various types of datasets. The Sunway workstation is equipped with a single SW26010-Pro processor, 96 GB of RAM, and runs UnionTech OS with SWGCC 7.1.0 as the compiler. The x86 Intel server features two Intel Platinum 8260 processors, 256 GB of RAM, and operates on Rocky Linux with GCC 9.4.0 for compilation. Both platforms are equipped with high-speed SSDs to reduce the impact of I/O speed on software performance. Table 2 presents the datasets used in the tests, covering different file sizes, sequence lengths, and sequencing technologies, all of which are freely available on the NCBI website.

Table 2. Dataset information.

NCBI Accession	#Reads	Read Length	Sequencing Technology	File Size
SRR2496709 ^a	12,607,411	100bp	DNA targeted sequencing	6.6 GB
SRR12131337 ^b	20,502,735	141bp	RNA sequencing	16.2 GB
SRR7963242 ^c	387,488,829	150bp	Whole genome sequencing	280.0 GB

^a <https://www.ncbi.nlm.nih.gov/sra/?term=SRR2496709>

^b <https://www.ncbi.nlm.nih.gov/sra/?term=SRR12131337>

^c <https://www.ncbi.nlm.nih.gov/sra/?term=SRR7963242>

3.1 Performance Comparison of Different SWBWA Versions

Figure 4 shows the performance of SWBWA across different optimization stages using the SRR2496709 dataset on the Sunway workstation. The single-threaded MPE version takes 25,145s to complete (2,921s for the seeding phase and 22,224s for the extension phase); however, for better readability, this result is not shown in the figure. In Fig. 4, the red line represents the speedup of each optimized version compared to the MPE version, while the bar chart compares the execution times and highlights the speedup achieved by each version over its predecessor. Starting from the “remove barrier” version, the seeding and extension phases are merged, making it impossible to measure their execution time separately; therefore, only the earlier versions display individual timings for these two phases. Additionally, the file I/O time is not shown in the figure because it is executed asynchronously with computation, and the test platform is equipped with high-speed SSDs, which completely hides the I/O overhead. Overall, compared to the MPE version, the optimized SWBWA achieves a $330\times$ speedup, fully leveraging the computational power of the multi-CPE architecture and the high-speed LDM resources of the Sunway platform.

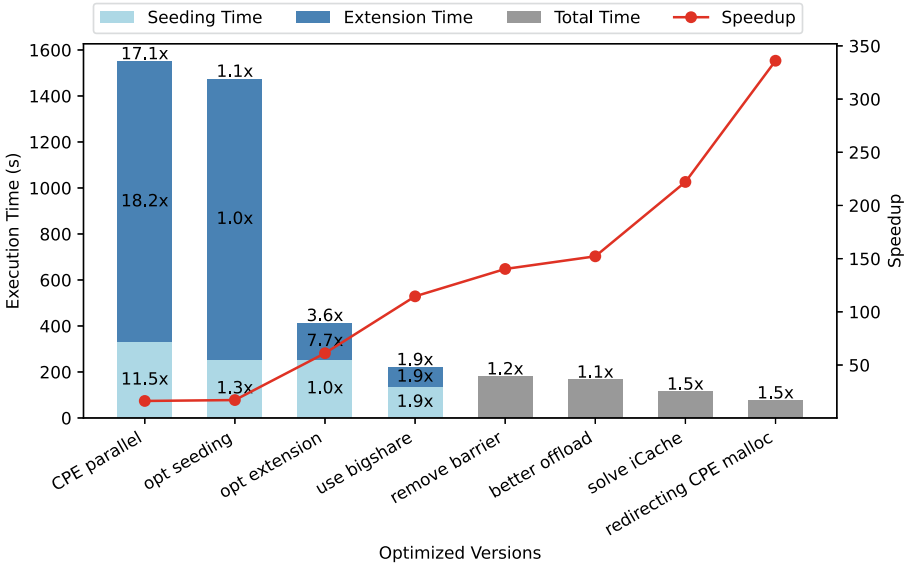


Fig. 4. Comparison of different SWBWA versions. The red line indicates the speedup over the MPE version, and the numbers on the bars show the speedup over the preceding version. (Color figure online)

From Fig. 4, we can observe that the “CPE parallel” version, which utilizes 64 CPEs for parallel acceleration, achieves less than a $20\times$ speedup. This limited parallel efficiency is mainly due to the random memory accesses to the FM-index

and the frequently used SSW computation matrix, both stored in main memory. On Sunway platform, the bandwidth for CPEs directly accessing main memory is highly limited, which restricts scalability across multiple CPEs. Therefore, in subsequent versions, we optimize random memory accesses and maximize the utilization of LDM wherever possible. The “opt seeding” version improves FM-index access by employing software prefetching and reducing redundant memory accesses, achieving a 30% performance improvement. The “opt extension” version optimizes LDM utilization and incorporates CPE vectorization for the SSW algorithm, achieving a significant $7.7\times$ speedup. The “use bigshare” version utilizes all six CGs but achieves only around a $2\times$ speedup compared to the previous version. This is mainly due to the amplification of load imbalance when scaling from 64 CPEs to 384 CPEs in bigshare mode, as well as performance constraints related to CPE iCache misses and private memory access (discussed in detail in Sect. 2.3). The following four versions progressively address these issues, reducing the average iCache miss time to around 8%, and ultimately achieving a $5.7\times$ speedup in bigshare mode compared to the single-CG version.

Table 3. Runtime comparison of SWBWA and BWA-MEM on different datasets.

Dataset	Software	Runtime (s)	Speedup	Memory Usage (GB)
SRR2496709	SWBWA	74.83	1.23x	11.24
	BWA-MEM	92.41	-	10.78
SRR12131337	SWBWA	150.49	1.43x	10.80
	BWA-MEM	214.47	-	10.41
SRR7963242	SWBWA	7353.42	1.24x	14.01
	BWA-MEM	9125.60	-	13.89

3.2 Comparison with X86 Platform

We compare the performance of SWBWA with BWA-MEM, the industry gold standard for NGS alignment on an x86 platform, which is widely validated and serves as a reliable benchmark. Our evaluation focuses solely on speed rather than accuracy since SWBWA produces nearly identical results to BWA-MEM. The minor differences arise from the statistical inference of insert size distribution in interleaved alignment regions, which can vary with different block sizes. Even BWA-MEM itself yields slightly different results depending on block size settings. When using the same block size or processing single-end data that does not involve insert size distribution inference, SWBWA generates output identical to BWA-MEM. Additionally, several optimized versions of BWA-MEM exist on the x86 platform, such as BWA-MEM2 [17], which improves performance by adjusting FM-index compression levels. However, BWA-MEM2 has significantly higher memory requirements as it relies on an alternative indexing strategy and

requires rebuilding the index which would exceed the main memory capacity of the Sunway architecture. In contrast, SWBWA can directly use the index built by BWA-MEM while maintaining nearly the same memory footprint (see Table 3).

Table 3 presents the performance comparison between SWBWA and BWA-MEM. SWBWA runs on the Sunway workstation, while BWA-MEM runs on a 48-core x86 server. The results demonstrate that SWBWA achieves a notable speedup across various types of sequencing data, with an average speedup of $1.3\times$.

4 Conclusion

In this paper, we propose SWBWA, the first efficient implementation of BWA-MEM on the Sunway platform. By fully adapting it to the Sunway architecture and leveraging multi-CPE parallelism, LDM, and vector instructions, SWBWA achieves over $300\times$ speedup compared to the unoptimized MPE version and delivers consistent performance improvements over BWA-MEM on an x86 platform. SWBWA further enriches the software ecosystem for sequencing data analysis on the Sunway platform, representing a key milestone in our comprehensive NGS pipeline deployment plan. Furthermore, by addressing two major performance bottlenecks in bigshare mode, we establish an efficient, MPI-free software migration approach on Sunway workstations. This enables porting of software originally designed for single-node x86 environments without requiring MPI adaptations, thereby also contributing to the development of the Sunway software ecosystem. Source code is written in C++ and publicly available at <https://github.com/RabbitBio/SWBWA>.

For future work, we plan to extend SWBWA to a multi-node version to fully exploit the computational resources of the Sunway supercomputing platform for large-scale biological data analysis. Additionally, we plan to encapsulate our optimizations for bigshare mode into a library or loader, enabling performance improvements without modifying program source code.

Acknowledgments. This work is partially supported by NSFC Grants 62102231; Shandong Provincial Natural Science Foundation (ZR2021QF089); Engineering Research Center of Digital Media Technology, Ministry of Education, China; DFG RMaP project C01.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Daily, J.: Parasail: SIMD C library for global, semi-global, and local pairwise sequence alignments. *BMC Bioinform.* **17**, 1–11 (2016)

2. Duan, X., et al.: S-aligner: Ultrascable read mapping on Sunway Taihu light. In: 2017 IEEE International Conference on Cluster Computing (CLUSTER), pp. 36–46. IEEE (2017)
3. Edgar, R.: Syncmers are more sensitive than minimizers for selecting conserved k-mers in biological sequences. *PeerJ* **9**, e10805 (2021)
4. Fu, H., Liao, J., Yang, J.o.: The Sunway Taihulight supercomputer: system and applications. *Sci. China Inf. Sci.* **59**, 1–16 (2016)
5. Jain, C., Diltthey, A., Koren, S., Aluru, S., Phillippy, A.M.: A fast approximate algorithm for mapping long reads to large reference databases. *J. Comput. Biol.* **25**(7), 766–779 (2018)
6. Kosugi, S., Momozawa, Y., Liu, X., Terao, C., Kubo, M., Kamatani, Y.: Comprehensive evaluation of structural variation detection algorithms for whole genome sequencing. *Genome Biol.* **20**, 1–18 (2019)
7. Li, H.: Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. arXiv preprint [arXiv:1303.3997](https://arxiv.org/abs/1303.3997) (2013)
8. Li, H.: Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics* **34**(18), 3094–3100 (2018)
9. Müller, A., Schmidt, B., Membarth, R., Leiße, R., Hack, S.: ANYSEQ/GPU: a novel approach for faster sequence alignment on GPUS. In: Proceedings of the 36th ACM International Conference on Supercomputing, pp. 1–11 (2022)
10. Roberts, M., Hayes, W., Hunt, B.R., Mount, S.M., Yorke, J.A.: Reducing storage requirements for biological sequence comparison. *Bioinformatics* **20**(18), 3363–3369 (2004)
11. Rucci, E., Garcia, C., Botella, G., De Giusti, A.E., Naiouf, M., Prieto-Matias, M.: OSWALD: OpenCL smith-waterman on Altera’s FPGA for large protein databases. *Int. J. High Perform. Comput. Appl.* **32**(3), 337–350 (2018)
12. Sahlin, K.: Effective sequence similarity detection with strobemers. *Genome Res.* **31**(11), 2080–2094 (2021)
13. Sahlin, K.: Strobealign: flexible seed size enables ultra-fast and accurate read alignment. *Genome Biol.* **23**(1), 260 (2022)
14. Schmidt, B., Hildebrandt, A.: Next-generation sequencing: big data meets high performance computing. *Drug Discov. Today* **22**(4), 712–717 (2017)
15. Schmidt, B., Kallenborn, F., Chacon, A., Hundt, C.: Cudasw++ 4.0: ultra-fast GPU-based smith–Waterman protein sequence database search. *BMC Bioinform.* **25**(1), 342 (2024)
16. Smith, T.F., Waterman, M.S., et al.: Identification of common molecular subsequences. *J. Mol. Biol.* **147**(1), 195–197 (1981)
17. Vasimuddin, M., Misra, S., Li, H., Aluru, S.: Efficient architecture-aware acceleration of bwa-mem for multicore systems. In: 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 314–324. IEEE (2019)
18. Xu, K., Duan, X., Müller, A., Kobus, R., Schmidt, B., Liu, W.: FMapper: scalable read mapper based on succinct hash index on Sunway Taihulight. *J. Parallel Distrib. Comput.* **161**, 72–82 (2022)
19. Yan, L., et al.: SWQC: efficient sequencing data quality control on the next-generation Sunway platform. *Futur. Gener. Comput. Syst.* **164**, 107577 (2025)
20. Zhu, Q., Luo, H., Yang, C., Ding, M., Yin, W., Yuan, X.: Enabling and scaling the HPCG benchmark on the newest generation sunway supercomputer with 42 million heterogeneous cores. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–13 (2021)